

React Node Express MERN FAQ

react js

25 most asked interview question for each
with there detailed interview answer with scenario and
coding example

also include small small coding question

context :preparing for capgemini exceller interview

React — 25 most-asked interview questions (Capgemini Exceller prep)

Nice — I made **25 core React questions** you'll commonly see in interviews. For each: a **short answer**, **quick deeper explanation**, a compact **coding example**, a one-line **scenario** for when it matters, and a tiny **coding exercise** you can practice immediately. Read them fast, practice the mini-coding tasks, and you'll be interview-ready.

Tip: during answers, mention trade-offs and why you chose a particular approach — interviewers care about judgment.

1) What is React?

Short: A UI library for building component-based user interfaces (by Facebook).

Explain: Declarative, component-driven, uses a Virtual DOM to update UI efficiently.

Focuses on view layer only.

Code (minimal):

```
jsx
```

```
function Hello({name}) { return <h1>Hello, {name}</h1>; }
```

Scenario: Use when building interactive front-ends with reusable UI pieces.

Mini task: Create a `<Greeting name="Chetan" />` component that shows `Hello, Chetan`.

2) What is JSX?

Short: Syntax extension that looks like HTML inside JS — compiled to

`React.createElement`.

Explain: JSX is optional but common — allows embedding expressions `{}` and components as tags. Beware of `className`, `{}` expressions, and single parent wrapper.

Code:

```
jsx

const el = <div className="card">{user.name}</div>;
```

Scenario: Writing component templates.

Mini task: Write JSX to render a list of three colors from an array.

3) Functional vs Class components

Short: Functional: functions + hooks; Class: ES6 classes with lifecycle methods. Hooks favor functional components now.

Explain: Hooks let functional components have state & lifecycle; class components still used but less common.

Code (hook vs class):

```
jsx

// Functional
function Counter(){ const [n,setN]=React.useState(0); return <button onClick={()=>setN(n+1)}>{n}
</button>; }
```

Scenario: Prefer functional with hooks for new code.

Mini task: Convert a small class-based counter to a functional component using `useState`.

4) Props vs State

Short: Props are read-only inputs to a component; state is internal, mutable data.

Explain: Props come from parent; state managed inside component (or via external store). Changing state triggers re-render.

Code:

jsx

```
function User({name}){ const [editing,setEditing]=React.useState(false); return <div>{editing?<input defaultValue={name}/> : name}</div>; }
```

Scenario: Use props to parametrize child; state for UI changes.

Mini task: Make a component that toggles text between "ON"/"OFF" using state.

5) Lifecycle methods & hooks equivalents

Short: `componentDidMount` → `useEffect(() => {}, [])`; `componentDidUpdate` → `useEffect(() => {}, [deps])`; `componentWillUnmount` → cleanup function in `useEffect`.

Explain: Hooks consolidate lifecycle logic; separate effects by dependency arrays.

Code:

jsx

```
useEffect(()=>{ const id = setInterval(tick,1000); return ()=> clearInterval(id); }, []);
```

Scenario: fetching data, timers, subscriptions.

Mini task: Use `useEffect` to fetch data once on mount (simulate with `setTimeout`).

6) `useState` basics & lazy init

Short: Hook for component state; supports lazy initializer function.

Explain: `const [s,setS]=useState(()=> expensiveInit())` runs initializer once.

Avoid complex updates without functional update when depending on previous state.

Code:

jsx

```
const [count,setCount] = useState( ()=> Number(localStorage.getItem('count') || 0) );
```

Scenario: counters, toggles, form fields.

Mini task: Implement a button that increments a number and persists to `localStorage` on change.

7) `useEffect` gotchas and cleanup

Short: Effects run after render; dependencies control re-run; return a cleanup for unmount or before re-run.

Explain: Missing deps → stale closures or excessive re-runs. Use ESLint plugin to detect missing deps. For subscriptions always cleanup.

Code:

jsx

```
useEffect(()=>{ window.addEventListener('resize', onResize); return ()=>
window.removeEventListener('resize', onResize); }, []);
```

Scenario: event listeners and subscriptions.

Mini task: Create `useEffect` that logs "mounted" once, and "unmounted" on cleanup.

8) `useRef` and accessing DOM

Short: `useRef` stores a mutable `.current` value that persists across renders (commonly used for DOM refs).

Explain: Changing `.current` doesn't trigger re-render. You can also store previous props/state. `forwardRef` passes refs down.

Code:

jsx

```
function Input(){ const ref = useRef(); return <input ref={ref} />; }
```

Scenario: focusing inputs, reading DOM size.

Mini task: Autofocus an input on mount using `useRef` and `useEffect`.

9) Controlled vs Uncontrolled components (forms)

Short: Controlled: React state holds input value; Uncontrolled: DOM holds value accessed via ref.

Explain: Controlled gives full control & validation; uncontrolled can be simpler for simple forms or performance-critical fields.

Code (controlled):

jsx

```
function NameForm(){ const [name,setName]=useState(""); return <input value={name}
onChange={e=>setName(e.target.value)}>; }
```

Scenario: Complex validations → controlled; quick file upload → uncontrolled.

Mini task: Build a controlled text input that blocks digits.

10) Keys in lists & reconciliation

Short: `key` prop helps React match list items between renders; use stable unique ids (not index ideally).

Explain: Wrong keys cause incorrect DOM reuse and bugs. Keys must be stable and unique per sibling list.

Code:

jsx

```
items.map(item => <li key={item.id}>{item.text}</li>)
```

Scenario: dynamic lists (add/remove/reorder).

Mini task: Show a list of tasks and delete one; ensure keys use id, not index.

11) Event handling & Synthetic events

Short: React wraps browser events in `SyntheticEvent` for cross-browser consistency; use camelCase `onClick`.

Explain: Synthetic events are pooled — don't use event after async without copying (e.g., `e.persist()` or extract values). Pass parameters via arrow functions or `bind`.

Code:

jsx

```
<button onClick={(e)=> handle(e, id)}>Click</button>
```

Mini task: Implement a button that logs the click event type and button id.

12) Context API

Short: Provides a way to pass data through the component tree without props drilling.

`createContext` , `Provider` , `useContext` .

Explain: Good for global theme, auth. Avoid overusing for frequently changing data — can cause re-renders. Memoize provider value.

Code:

jsx

```
const ThemeCtx = React.createContext('light');
function App(){ return <ThemeCtx.Provider value="dark"><Child/></ThemeCtx.Provider> }
function Child(){ const theme = useContext(ThemeCtx); return <div>{theme}</div>; }
```

Scenario: theme, locale, auth user.

Mini task: Create a theme context and toggle between "light"/"dark" in children.

13) Higher-Order Components (HOC) vs Render Props

Short: HOC: function that returns enhanced component; Render Props: component that accepts a function child to provide data. Both for code reuse.

Explain: HOCs wrap components (e.g., `withAuth`); render props pass render-control to children. Hooks often replace both now.

Code (HOC):

jsx

```
function withLogger(Wrapped){ return props => { console.log('render'); return <Wrapped
{...props}/> } }
```

Mini task: Write a HOC `withTimestamp` that passes `timestamp` prop to wrapped component.

14) Rules of Hooks

Short: Call hooks only at top-level and only from React functions (not loops, conditions, or nested functions).

Explain: Rules ensure consistent hook order across renders; breaking them breaks state mapping. Use custom hooks to reuse logic.

Mini task: Convert repeated `useEffect` + `useState` logic into a custom hook `useWindowWidth`.

15) `useMemo` vs `useCallback`

Short: `useMemo` memoizes computed values; `useCallback` memoizes functions (returns stable reference).

Explain: Use to avoid expensive recalcs or to prevent child re-renders when passing functions. Don't overuse — measure first.

Code:

```
jsx

const expensive = useMemo(() => heavyCalc(data), [data]);
const handle = useCallback(() => setX(x+1), [x]);
```

Mini task: Use `useMemo` to compute filtered list only when `items` or `query` change.

16) `useReducer`

Short: Alternative to `useState` for complex state transitions; similar to Redux reducer pattern.

Explain: Good for form logic or state with multiple sub-values; easier to test.

Code:

```
jsx

function reducer(state, action) { switch(action.type) { case 'inc': return {count: state.count+1};
default: return state } }
const [state, dispatch] = useReducer(reducer, {count:0});
```

Mini task: Implement a simple counter with `useReducer` supporting 'inc' and 'dec'.

17) Virtual DOM & Reconciliation

Short: React creates a Virtual DOM diff to compute minimal DOM changes. Reconciliation uses keys to match elements.

Explain: Updates are batched/optimized. Understanding helps optimize rendering

patterns.

Mini task: Explain (in 2–3 sentences) why mutating array in place vs returning new array affects reconciliation.

18) Portals

Short: `ReactDOM.createPortal(children, container)` renders children into a DOM node outside parent hierarchy (useful for modals).

Explain: Keeps component logic intact but places markup elsewhere (e.g., top-level modal root). Events propagate through React tree normally.

Code:

jsx

```
return ReactDOM.createPortal(<div className="modal">{children}</div>,
document.getElementById("modal-root"));
```

Mini task: Build a modal component that mounts into `#modal-root` using a portal.

19) Error Boundaries

Short: React components that catch rendering errors in descendants (`componentDidCatch`) — implemented as class components.

Explain: Useful to show fallback UI and avoid entire app crash. Hooks don't provide error boundary API yet.

Code:

jsx

```
class ErrorBoundary extends React.Component {
  state={err:null};
  componentDidCatch(error){ this.setState({err:error}); }
  render(){ if(this.state.err) return <div>Something went wrong</div>; return this.props.children; }
}
```

Mini task: Wrap a component that throws inside `ErrorBoundary` and display fallback UI.

20) `React.lazy` & `Suspense`

Short: Lazy-load components to split bundles: `const Comp =`

`React.lazy(()=>import('./Comp'))` and wrap in `<Suspense fallback>`.

Explain: Improves initial load by code-splitting. For data fetching suspense support is evolving.

Code:

jsx

```
const Lazy = React.lazy(()=>import('./Big'));  
function App(){ return <Suspense fallback={<div>Loading</div>}><Lazy/></Suspense>; }
```

Mini task: Convert a heavy component to lazy and test loading fallback.

21) SSR vs CSR vs Hydration

Short: CSR: render in browser; SSR: render on server and send HTML; Hydration: attach React event handlers to server-rendered HTML.

Explain: SSR improves perceived load & SEO; hydration reuses server markup. Tools: Next.js for SSR/hybrid. Be mindful of code that uses `window` on server.

Mini task: Explain how you'd guard `window` usage in components to work with SSR.

22) Performance optimizations

Short: Memoize components (`React.memo`), avoid unnecessary re-renders (`useMemo`, `useCallback`), virtualize long lists, code-split, avoid heavy work on main thread.

Explain: Profile first (React DevTools). Batching updates, avoiding inline objects as props, and using proper keys help.

Mini task: Wrap a pure child component with `React.memo` and show how prop identity affects re-render.

23) Testing React components

Short: Use Jest + React Testing Library: test behavior (what user sees/does), not implementation.

Explain: Prefer `screen.getByText`, user events; avoid shallow-only tests for real

interactions. Snapshot tests for stable UI surfaces.

Code (example):

```
js

// Simple RTL test
render(<Button>Click</Button>);
fireEvent.click(screen.getByText('Click'));
expect(handler).toHaveBeenCalled();
```

Mini task: Write a test that verifies clicking a button toggles text from "OFF" to "ON".

24) Accessibility (a11y) basics in React

Short: Use semantic HTML (`<button>` , `<label>`), ARIA where necessary, and keyboard support. Tools: axe, Lighthouse.

Explain: Provide `alt` for images, manage focus for modals, ensure keyboard navigation and proper contrast. Accessibility is often asked in interviews.

Mini task: Build an accessible modal with focus trap and `aria-modal="true"`.

25) State management: Context vs Redux vs others

Short: Small apps: Context + hooks. Complex apps: Redux/RTK or MobX when you need predictable, traceable state and middleware. Newer patterns: React Query for server state.

Explain: Distinguish *local UI state* vs *global/app state* vs *server state*. Choose simplest tool that meets requirements.

Mini task: Implement a counter using Context + `useReducer` shared across two sibling components.

Quick “small coding question” set (practice)

1. Create a functional counter with increment/decrement and disable decrement at 0.
2. Build a controlled input that only accepts lowercase letters.
3. Render a dynamic list and add/remove items; preserve item order with stable `id` keys.
4. Make a `useToggle` custom hook and use it in a component.

5. Lazy-load a component and show a spinner during load.
 6. Write a simple `ErrorBoundary` class and test it with a component that throws on a button click.
 7. Implement `usePrevious` hook to access previous prop value.
 8. Create a form with validation (name required, email pattern) and show errors.
 9. Build a `ThemeContext` to toggle dark/light and persist preference in `localStorage`.
 10. Virtualize a list (or explain approach) if asked about very large lists.
-

How to use this sheet for Capgemini Exceller interview

1. **Practice explaining trade-offs** (why Context vs Redux, why memoization).
 2. **Write the small tasks** — 10–20 minutes each — and push to GitHub. Real code impresses.
 3. **Mock interview:** explain one concept in 2 minutes, then implement a tiny example live (e.g., `useEffect` fetch + cleanup).
 4. **Focus on clarity:** Capgemini interviewers like clear, structured answers: state → reason → example → exceptions.
-

If you want, I can now:

- Convert these into a printable one-page cheat sheet, or
- Generate **complete solutions** (full code + tests) for any **5** of the mini-tasks above (pick which 5).

Which of the two would help you most right now?

same for node js

Node.js — 25 most-asked interview questions (Capgemini Exceller prep)

Great — here's the **Node.js** version of the React sheet: **25 common interview questions**, each with a **short answer**, a **brief deeper explanation**, a tiny **coding example**, a one-line **scenario** when it matters, and a **mini coding task** you can practice

in 5–20 minutes. Use these to speak clearly, show trade-offs, and write a quick demo during interviews.

Tip: for Capgemini interviews show: *what* something is → *why it matters* → *one small example* → *trade-offs / gotchas*.

1) What is Node.js?

Short: Node.js is a JavaScript runtime built on Chrome's V8 that runs JS on the server.

Explain: Event-driven, non-blocking I/O model — ideal for I/O-heavy apps. Not a framework; it's a runtime.

Code:

```
js

// hello.js
console.log('Hello from Node', process.pid);
```

Scenario: Building REST APIs, real-time apps (chat), CLI tools.

Mini task: Run `node hello.js` and print current working directory with `process.cwd()`.

2) Event Loop — what is it and how does it work?

Short: The event loop processes callbacks, timers, I/O callbacks, and microtasks (Promises) to keep Node non-blocking.

Explain: Has phases (timers, pending callbacks, idle, poll, check, close). Microtasks (Promise callbacks) run between phases. Blocking JS stalls the loop.

Code:

```
js

setTimeout(()=>console.log('timer'),0);
Promise.resolve().then(()=>console.log('promise'));
console.log('sync');
```

Output order: `sync`, `promise`, `timer`.

Mini task: Create script showing order of `setImmediate`, `setTimeout(...,0)`, and a

resolved Promise.

3) Callback vs Promise vs async/await

Short: Callbacks are functions passed to async ops; Promises provide composable async values; `async/await` is syntactic sugar over Promises.

Explain: Use Promises/async-await for readability; handle errors with `try/catch`. Avoid callback hell by converting to Promises.

Code:

```
js

async function getData(){ try { const r = await fetch(url); return await r.json(); } catch(e){ throw e } }
```

Scenario: Fetch DB rows, call external APIs.

Mini task: Convert a callback-based `fs.readFile` example to `fs.promises.readFile` with `async/await`.

4) How does `require` work? CommonJS vs ES Modules

Short: `require()` loads CommonJS modules synchronously; ES modules use `import / export` and are asynchronous/hoisted differently.

Explain: Node supports both but with rules (`"type": "module"` in package.json or `.mjs` extension). CommonJS uses `module.exports` object.

Code:

```
js

// commonjs
module.exports = { sum: (a,b)=>a+b };

// es module
export const sum = (a,b)=>a+b;
```

Scenario: Choosing module type for library distribution.

Mini task: Create `math.js` using CommonJS and import it from `index.js`.

5) Explain Streams (Readable/Writable/Transform)

Short: Streams handle large/chunked data (files, network) without loading it all into memory. Types: Readable, Writable, Duplex, Transform.

Explain: Use `.pipe()` to connect streams for memory-efficient processing. Great for file uploads/downloads.

Code:

```
js

const fs = require('fs');
fs.createReadStream('big.txt').pipe(fs.createWriteStream('copy.txt'));
```

Scenario: Uploading large files or streaming video.

Mini task: Pipe an HTTP response to a file (`http.get(url, res => res.pipe(fs.createWriteStream('out')))`).

6) Buffer vs String

Short: `Buffer` is raw binary data; strings are text (UTF-8 by default). Buffers used for binary I/O.

Explain: Convert via `Buffer.from('text')` and `.toString()`. Streams emit `Buffer` objects unless encoding set.

Code:

```
js

const b = Buffer.from('hello');
console.log(b.toString('utf8'));
```

Mini task: Read a binary file and log first 8 bytes in hex.

7) EventEmitter

Short: Base class for emitting/listening to events. Many Node APIs extend it.

Explain: Use `.on()`, `.once()`, `.emit()`. Remove listeners to avoid leaks.

Code:

```
js
```

```
const EventEmitter = require('events');
const ee = new EventEmitter();
ee.on('ping', payload=>console.log('pong', payload));
ee.emit('ping','here');
```

Scenario: Implementing pub/sub within an app.

Mini task: Build a simple counter emitter that emits `'tick'` every second and stops after 5 ticks.

8) Non-blocking I/O vs Blocking

Short: Node's core is non-blocking for I/O; CPU-bound work blocks event loop.

Explain: Use worker threads / child processes for heavy computation. Prefer async fs/network calls.

Mini task: Demonstrate `fs.readFileSync` blocking by logging timestamps before/after and compare to async version.

9) Error handling patterns

Short: For async/await use `try/catch`; for callbacks first param is `err`; unhandled rejections (`process.on('unhandledRejection')`) need attention.

Explain: Always validate inputs; centralize error middleware in Express. For critical errors consider process restart.

Code (Express error middleware):

```
js

app.use((err, req, res, next) => { console.error(err); res.status(500).json({ error: err.message }); });
```

Mini task: Create an async route that throws and handle with Express error middleware.

10) process object & environment variables

Short: `process` gives runtime info (`process.env`, `process.pid`, signals). Use env vars for config.

Explain: Use `dotenv` in dev to load `.env`. Never commit secrets.

`process.on('SIGINT', ...)` for graceful shutdown.

Mini task: Add SIGINT handler that closes a mock server before exiting.

11) Child processes and Worker Threads

Short: `child_process` spawns new processes; `worker_threads` share memory and are better for CPU tasks.

Explain: Use `spawn` for streaming I/O, `exec` for buffered output. Use workers for CPU-bound tasks to avoid blocking loop.

Code (spawn):

```
js

const { spawn } = require('child_process');
const ls = spawn('ls', ['-la']);
ls.stdout.on('data', d => console.log(d.toString()));
```

Mini task: Spawn `node -v` and print its stdout.

12) npm / package.json basics

Short: `package.json` declares metadata and scripts. Use `npm install` to add deps; `package-lock.json` locks versions.

Explain: Prefer `npm ci` in CI; semantic versioning matters (`^` vs `~`). Use `scripts` for common tasks.

Mini task: Create a `package.json` with `"start": "node index.js"` and run with `npm start`.

13) How to structure a Node project

Short: Keep `src/`, `routes/`, `controllers/`, `services/`, `models/`, and `tests/`. Use config per environment.

Explain: Separation of concerns improves testability and clarity. Avoid huge monolithic files.

Mini task: Sketch (create folder + placeholder files) a small Express CRUD project structure.

14) Express.js basics — middleware, routing

Short: Express uses middleware functions (`( req, res, next )`) that run in order; routers modularize endpoints.

Explain: Middleware can be global or route-specific. Use `next(err)` to delegate to error handlers.

Code:

```
js

const express = require('express');
const app = express();
app.use(express.json());
app.get('/ping', (req, res) => res.send('pong'));
app.listen(3000);
```

Mini task: Write middleware that logs method and URL for each request.

15) CORS and common networking issues

Short: CORS is browser-enforced policy; server must set `Access-Control-Allow-*` headers. Use `cors` middleware.

Explain: Not a Node concept; fix via server headers or proxy. Also know keep-alive, timeouts, and connection limits.

Mini task: Enable CORS for all origins in a simple Express app using the `cors` package.

16) REST API design & status codes

Short: Use proper verbs and status codes (200 OK, 201 Created, 204 No Content, 400 Bad Request, 401/403, 404, 500). Keep endpoints resource-oriented.

Explain: Validate input, return consistent error shape, and implement pagination for lists.

Mini task: Implement `POST /users` that returns `201` with created user id.

17) Security basics — input validation, sanitization, helmet

Short: Validate user input, escape outputs, use `helmet`, rate-limit, and avoid `eval` / `child_process` with user input.

Explain: Protect against injection, XSS in server-generated HTML, and DoS. Secure

configuration and secrets management matter.

Mini task: Add `helmet()` and a simple rate limiter to an Express app.

18) Logging & monitoring

Short: Use structured logging (winston/pino), centralize logs, and add health checks and metrics (Prometheus).

Explain: Use log levels, rotate logs, and avoid logging secrets. Implement graceful restart and liveness/readiness endpoints.

Mini task: Add `pino` logger and log an object `{event: 'user_signup', userId}`.

19) Caching strategies

Short: Cache at multiple layers: in-memory (LRU), reverse proxy (Varnish/Cloud CDN), Redis for shared cache.

Explain: Cache invalidation is hard — choose TTLs and cache keys carefully. Use ETag for conditional responses.

Mini task: Implement a simple memory cache for expensive function results with TTL.

20) Database connectivity patterns (Mongo/Postgres)

Short: Use connection pools, parameterized queries/ORMs (Sequelize, TypeORM, Prisma, Mongoose). Close connections gracefully.

Explain: Use transactions for multi-step writes; favor prepared statements to prevent injection.

Mini task: Connect to a local SQLite or in-memory DB and run a simple INSERT + SELECT.

21) Testing Node apps (unit & integration)

Short: Use Jest/Mocha for unit tests; supertest for API integration tests. Test behavior, not implementation.

Explain: Mock external services, isolate DB or use test DB, and run tests in CI.

Mini task: Write a Jest test that hits an Express GET route and expects 200.

22) Debugging Node.js

Short: Use `node --inspect` + Chrome DevTools or VS Code debugger; use `console`, logs, and `--trace-warnings`.

Explain: Inspect heap snapshots for memory leaks, CPU profiles for hotspots. Use `nodemon` for dev reloads.

Mini task: Run `node --inspect-brk script.js` and connect via Chrome `chrome://inspect`.

23) Memory leaks & profiling

Short: Leaks often from global caches, forgotten timers, event listeners. Use heap snapshots and `clinic / 0x` to profile.

Explain: Watch for growth over time, analyze retained objects and closures holding memory.

Mini task: Create a closure that accidentally retains an array; observe memory growth with simple repeated calls.

24) Scaling Node applications

Short: Use process managers (PM2), clustering (`cluster` or node's `worker_threads`), container orchestration (Kubernetes), and load balancers.

Explain: Node is single-threaded; cluster spawns multiple processes to use all CPU cores. Prefer horizontal scaling with stateless services and external session store.

Mini task: Spin up a basic cluster that forks workers equal to CPU count and prints worker id handling requests.

25) Best practices & common pitfalls

Short: Don't block the event loop; handle errors centrally; validate inputs; secure secrets; test; use linting and type checking (TypeScript).

Explain: Use consistent folder layout, small modules, CI pipelines, and monitor production. Prefer simple, measured optimizations.

Mini task: Add ESLint + Prettier + a `lint` script to `package.json` and fix one lint warning.

Quick “small coding question” set (practice problems)

1. Create an HTTP server (native `http`) that responds with JSON `{uptime, pid}`.
 2. Write an Express route that validates body fields and returns 400 on missing fields.
 3. Implement a rate limiter in-memory (per IP, 100 req/min).
 4. Stream a large file to client with range requests support (partial content).
 5. Implement a simple in-memory LRU cache class.
 6. Spawn a child process to run a shell command and stream stdout to client.
 7. Write a function that retries a failing Promise N times with exponential backoff.
 8. Build a CLI using `commander` that accepts options and prints formatted output.
 9. Create a script that watches a directory and logs file changes (`fs.watch`).
 10. Implement pagination for a large array: `GET /items?page=2&limit=10`.
 11. Make a small worker using `worker_threads` that computes Fibonacci for large N.
 12. Create middleware that authenticates a JWT from `Authorization` header.
 13. Write a health-check endpoint returning `200` only when DB connection is healthy (mock DB).
 14. Implement graceful shutdown: stop accepting new requests, wait for current ones, close connections.
 15. Unit test a controller function using Jest, mocking DB calls.
-

How to use this sheet for Capgemini Exceller interview

1. **Practice 6–8 of the mini-tasks** and push them to GitHub — interviewers like samples.
 2. **Be ready to explain trade-offs** (e.g., why use worker threads vs cluster; why Redis vs local cache).
 3. **Show awareness of production concerns**: logging, monitoring, graceful shutdown, security.
 4. **Prepare one architecture**: design a scalable chat or file-upload service — draw flow and pick tools.
-

Want me to:

- Convert these into a **one-page printable cheat sheet**, or
- Generate **complete code solutions** (with README and tests) for any **5** of the mini-tasks above (pick which 5)?

Pick one and I'll produce code + instructions right away.

same for express js

Express.js — 25 most-asked interview questions (Capgemini Exceller prep)

Nice — here's the **Express.js** cheat-sheet you asked for: **25 commonly asked interview questions**, each with a **short answer**, a **brief explanation**, a compact **coding example**, a one-line **scenario** when it matters, and a tiny **mini-task** you can practice in 5–20 minutes. Practice the mini-tasks and be ready to explain trade-offs and production considerations — Capgemini interviewers like clarity and practical thinking.

Quick tip: structure answers in interviews as **(What)** → **(Why it matters)** → **(Example)** → **(Trade-offs/Gotchas)**.

1) What is Express.js?

Short: A minimal, flexible Node.js web framework for building web/apps and APIs.

Explain: Sits on top of Node's `http` module; provides routing, middleware, and utilities to build servers quickly. Lightweight and unopinionated.

Code:

```
js

const express = require('express');
const app = express();
app.get('/ping', (req,res) => res.json({pong: true}));
app.listen(3000);
```

Scenario: Quickly build REST APIs or small web backends.

Mini task: Create `GET /hello` that returns `{hello: "Chetan"}`.

2) What is middleware in Express?

Short: Functions that sit between request and response, with signature `(req, res, next)`.

Explain: Middleware can read/modify req/res, end the request, or call `next()` to pass control. Types: application, router, error, and third-party middleware.

Code:

```
js

app.use((req,res,next)=>{ console.log(req.method, req.url); next(); });
```

Scenario: Logging, auth, parsing bodies, error handling.

Mini task: Write middleware that rejects requests without header `x-api-key`.

3) Order of middleware — why does it matter?

Short: Express executes middleware in registration order; placement affects behavior.

Explain: For example, authentication should run before route handlers; error middleware must be last. Misordered middleware causes bugs or security holes.

Code:

```
js

app.use(authMiddleware); // must come before protected routes
app.get('/private', (req,res)=>res.send('ok'));
```

Scenario: Protect endpoints; ensure body parsing before route uses `req.body`.

Mini task: Show a failing case if `app.use(express.json())` is placed after routes that expect `req.body`.

4) How do you parse request bodies?

Short: Use built-in `express.json()` and `express.urlencoded()` (or body-parser for legacy).

Explain: `express.json()` parses JSON bodies; `urlencoded` handles form data. Multipart (file uploads) needs `multer`.

Code:

```
js
```

```
app.use(express.json());
app.post('/user', (req,res)=> res.json({received: req.body}));
```

Scenario: Accept POST/PUT payloads in APIs.

Mini task: Implement `POST /login` that reads `{username,password}` and responds 400 if missing.

5) Routing basics & route parameters

Short: Define handlers for HTTP verbs and paths. Use `:param` for path params and `req.params` / `req.query` / `req.body`.

Explain: Routes can be modularized using `express.Router()`. Order and specificity matter (more specific routes before param routes).

Code:

```
js

const router = express.Router();
router.get('/users/:id', (req,res)=> res.json({id: req.params.id}));
app.use('/api', router);
```

Scenario: Resourceful APIs (e.g., `/users/:id`).

Mini task: Create `GET /api/products?page=2&limit=10` and log page+limit.

6) Error handling in Express

Short: Error middleware has four args (`err, req, res, next`) and must be last. Use `next(err)` to forward errors.

Explain: Centralized error handling simplifies response format and logging. Differentiate client (4xx) vs server (5xx) errors.

Code:

```
js

app.use((err,req,res,next)=> {
  console.error(err);
  res.status(err.status || 500).json({error: err.message});
});
```

Scenario: Return consistent error JSON and log stack traces.

Mini task: Throw an error in a route and handle it with error middleware.

7) Async route handlers & avoiding unhandled rejections

Short: Use `async` functions and catch errors to call `next(err)` (or use a wrapper).

Explain: Uncaught Promise rejections won't reach Express error middleware. Use a helper to wrap async handlers.

Code (wrapper):

```
js

const wrap = fn => (req,res,next)=> Promise.resolve(fn(req,res,next)).catch(next);
app.get('/data', wrap(async (req,res)=> { const d = await db.get(); res.json(d); }));
```

Scenario: DB calls, remote API calls inside routes.

Mini task: Implement and use an `asyncHandler` wrapper.

8) Serving static files

Short: Use `express.static()` to serve static assets from a directory.

Explain: Configure caching headers, set `maxAge`, and ensure `index.html` behavior as needed. For production, often use CDN or reverse proxy for static assets.

Code:

```
js

app.use(express.static('public', { maxAge: '1d' }));
```

Scenario: Serve SPA assets or images.

Mini task: Serve `public/index.html` and test by visiting

`http://localhost:3000/index.html`.

9) Templating engines

Short: Express supports view engines (EJS, Pug, Handlebars) via `app.set('view engine', ...)`.

Explain: Useful for server-rendered pages. For SPAs prefer APIs + client rendering.

Protect templates from XSS by escaping variables.

Code:

```
js

app.set('view engine', 'pug');
app.get('/', (req,res)=> res.render('index',{title:'Home'}));
```

Scenario: Simple server-rendered pages, emails, PDFs.

Mini task: Render a Pug template that lists 3 items.

10) Cookies & sessions

Short: Use `cookie-parser` to read cookies, and `express-session` (or Redis session store) for server sessions.

Explain: Sessions store state server-side; cookies can be signed/secure/HttpOnly. For stateless auth prefer JWTs. Don't store sensitive info in client cookies.

Code:

```
js

const session = require('express-session');
app.use(session({ secret: 's3cret', resave:false, saveUninitialized:false }));
app.get('/visits', (req,res)=> { req.session.v = (req.session.v||0)+1; res.send(String(req.session.v));
});
```

Scenario: Login sessions, cart state for small apps.

Mini task: Implement session visit counter.

11) Authentication patterns (JWT vs sessions)

Short: Sessions keep server state; JWTs are stateless tokens stored client-side. Use HTTPS for both.

Explain: JWTs scale well (no server state) but revocation is harder; sessions are easier to revoke but need shared store in multi-server setups. Consider refresh tokens, short lifetimes.

Code (JWT verify):

```
js
```

```
const jwt = require('jsonwebtoken');
function auth(req,res,next){
  const token = req.header('Authorization')?.split(' ')[1];
  if(!token) return res.sendStatus(401);
  try { req.user = jwt.verify(token, 'secret'); next(); } catch(e) { res.sendStatus(401); }
}
```

Scenario: Public APIs (JWT) vs sites requiring immediate revocation (sessions).

Mini task: Protect `/profile` route with a simple JWT check.

12) File uploads (multer)

Short: Use `multer` for multipart/form-data file uploads.

Explain: Multer handles storage (memory/disk) and file validation. For production upload to S3 or object storage rather than storing locally.

Code:

```
js

const multer = require('multer');
const upload = multer({ dest: 'uploads' });
app.post('/upload', upload.single('file'), (req,res)=> res.json({file: req.file}));
```

Scenario: Avatar/image upload endpoints.

Mini task: Upload a file via Postman and log `req.file`.

13) CORS — what and how to enable?

Short: CORS is a browser security policy; enable using `cors` package or response headers.

Explain: Configure allowed origins, methods, and credentials. Don't enable wide open CORS unless necessary.

Code:

```
js

const cors = require('cors');
app.use(cors({ origin: 'https://example.com', credentials: true }));
```

Scenario: Frontend on different domain calls your API.

Mini task: Enable CORS for `http://localhost:3000` only.

14) Rate limiting & brute force protection

Short: Use `express-rate-limit` or API gateway to throttle requests per IP.

Explain: Prevent abuse and DoS. Combine with IP reputation and caching; be careful with shared IPs (NAT).

Code:

```
js

const rateLimit = require('express-rate-limit');
app.use(rateLimit({ windowMs: 60*1000, max: 100 }));
```

Scenario: Public APIs, auth endpoints.

Mini task: Add rate limiting to `/login` to 5 attempts per minute.

15) Input validation & sanitization

Short: Use libraries like `Joi`, `zod`, or `express-validator` to validate and sanitize `req` inputs.

Explain: Prevent injection bugs and invalid data; validation centralizes contracts for endpoints. Return 4xx for invalid input.

Code (Joi):

```
js

const Joi = require('joi');
const schema = Joi.object({name: Joi.string().min(2).required()});
app.post('/user', (req,res)=> {
  const {error,val} = schema.validate(req.body);
  if(error) return res.status(400).json({error: error.details});
  // use val
});
```

Scenario: Any public endpoint that receives data.

Mini task: Validate `POST /register` requires `email` and `password` (min 6 chars).

16) Logging best practices

Short: Use structured loggers (winston, pino), include request IDs and levels, avoid logging secrets.

Explain: Integrate with centralized logging (ELK, Loki), add request correlation for tracing. Use middleware to attach request id.

Code:

```
js

const pino = require('pino')();
app.use((req,res,next)=> { req.log = pino.child({reqId: Date.now()}); next(); });
```

Scenario: Debugging production issues, audits.

Mini task: Log incoming request method/url and response time.

17) Testing Express apps

Short: Use Jest/Mocha + Supertest for route integration tests and unit tests for controllers/services.

Explain: Prefer testing behavior (status & body). Mock DB or use in-memory DB for integration.

Code (supertest):

```
js

const request = require('supertest');
const app = require('./app'); // express app
test('GET /ping', async ()=> {
  const res = await request(app).get('/ping');
  expect(res.status).toBe(200);
  expect(res.body).toEqual({pong:true});
});
```

Scenario: CI pipelines and regression prevention.

Mini task: Write a Supertest case for `POST /login` returning 400 for missing fields.

18) Static vs dynamic content — where to put logic?

Short: Keep heavy logic in services, controllers, or separate modules — routes should orchestrate.

Explain: Routes should be thin: validate → call service → return. This makes testing easier and separates concerns.

Mini task: Refactor a route so the DB call moves into `userService.create()`.

19) Performance tuning (compression, caching, gzip)

Short: Use `compression` for gzip, enable caching headers, and offload static content to CDNs. Profile with load testing.

Explain: Also avoid blocking sync calls, use connection pooling for DB, and chunk responses for large payloads.

Code:

```
js

const compression = require('compression');
app.use(compression());
```

Scenario: High-traffic APIs, large JSON responses.

Mini task: Add `compression()` and test response `Content-Encoding` header.

20) Streaming responses (large payloads)

Short: Use Node streams to send large data without buffering all in memory (e.g., `res.pipe()` from read stream).

Explain: Useful for file downloads, CSV exports, or server-sent events. Set proper headers for partial requests (range).

Code:

```
js

const fs = require('fs');
app.get('/download', (req,res)=> {
  const rs = fs.createReadStream('bigfile.zip');
  rs.pipe(res);
});
```

Scenario: File downloads, media streaming.

Mini task: Stream a local file to the client.

21) Graceful shutdown & signals

Short: Listen to process signals (`SIGINT` , `SIGTERM`) and stop accepting new requests, close server and DB connections.

Explain: Prevent data loss and ensure connections close cleanly before process exit. Important for containers/orchestration.

Code:

```
js

const server = app.listen(3000);
process.on('SIGTERM', async () => {
  server.close(() => { /* close DB */ process.exit(0); });
});
```

Scenario: Deployments, container restarts.

Mini task: Implement `SIGINT` handler that closes the server.

22) Mountable routers & modularization

Short: Use `express.Router()` to group routes by feature and mount them (`app.use('/users', usersRouter)`).

Explain: Encourages modular code, easier testing and maintenance. Keep controllers/services separate.

Mini task: Create `routes/users.js` that exports a router with `GET /` and mount it.

23) Integrating WebSockets (Socket.io) with Express

Short: Express serves HTTP, Socket.io or ws enables WebSockets; both can share the same underlying HTTP server.

Explain: Use Socket.io for rooms and fallback transports. Be mindful of scaling: sticky sessions or external pub/sub (Redis) required.

Code:

```
js
```

```
const http = require('http').createServer(app);
const io = require('socket.io')(http);
io.on('connection', socket => { socket.on('msg', m => io.emit('msg', m)); });
http.listen(3000);
```

Scenario: Real-time chat, notifications.

Mini task: Broadcast a message received over socket to all clients.

24) Security hardening (helmet, content security)

Short: Use `helmet`, validate inputs, rate limit, secure cookies, and avoid `eval` /unsafe deserialization.

Explain: Configure CSP, HSTS, XSS protections; store secrets in env and rotate keys. Regularly run dependency scans.

Code:

```
js

const helmet = require('helmet');
app.use(helmet());
```

Mini task: Add `helmet()` and show `X-Content-Type-Options` header presence.

25) Deployment patterns & best practices

Short: Build small stateless services, use process manager (PM2) or containers, externalize config via env vars, use health checks.

Explain: For scaling use multiple instances behind LB, external session store (Redis), CI/CD pipelines, and monitoring. Container images should be minimal and reproducible.

Mini task: Add `/health` endpoint returning `200` when server runs and `500` if a mock DB check fails.

Quick “small coding question” set (practice problems)

1. Create a simple Express `GET /status` returning `{uptime,pid}`.
2. Implement `POST /users` with Joi validation and return `201` with created user id.
3. Add logging middleware that logs method, path, status and response time.

4. Protect `/profile` with JWT middleware and write a token generator route for testing.
 5. Implement `POST /upload` using `multer` and save file metadata to an in-memory array.
 6. Add rate limiter to `/auth` (5 attempts per minute) using `express-rate-limit`.
 7. Create `usersRouter` with CRUD routes using `express.Router()` and mount under `/api/users`.
 8. Stream a large CSV file generated on the fly (use `Readable` stream) to the client.
 9. Add `compression()` and show that response is gzipped.
 10. Write an error middleware that returns `{error, code}` and test by throwing an error in a route.
 11. Implement graceful shutdown that finishes active requests then exits.
 12. Build a small integration test with `supertest` for `GET /status`.
 13. Add CORS only for `http://localhost:3000`.
 14. Create an Express app that serves SPA (`index.html`) for all unknown routes (support client-side routing).
 15. Implement a simple in-memory cache for `/expensive` endpoint with TTL.
 16. Use `express-session` with `connect-redis` (explain how to mock for local dev).
 17. Expose `/metrics` endpoint with simple request counters (simulate Prometheus metrics).
 18. Integrate Socket.io that emits a `time` event every second to connected clients.
 19. Implement `GET /files/:name` that streams files and supports range requests (partial content).
 20. Create middleware that enforces API version via `Accept` header and routes accordingly.
-

How to use this sheet for Capgemini Exceller interview

1. **Pick 8–10 mini tasks** from above and implement them in a small repo — push to GitHub. Interviewers like concrete examples.
 2. **Practice 2-minute explanations** for 6 core topics: middleware, error handling, async handlers, security, scaling, and testing.
 3. **Prepare one short architecture** (e.g., file-upload service): show endpoints, storage (S3), authFlow, scaling and monitoring. Use simple diagrams if asked.
 4. **During interviews** state trade-offs (e.g., JWT vs sessions, local file storage vs S3) — show you can reason beyond syntax.
-

If you want, I can now:

- Convert this into a **one-page printable cheat sheet**, or
- Generate **complete working code** (Express app + README + tests) for any 5 of the mini-tasks above (pick which 5).

Which would you like next?

same for mongo db

MongoDB — 25 most-asked interview questions (Capgemini Exceller prep)

Nice — here's the **MongoDB** version of your interview sheet: **25 common questions**, each with a **short answer**, a **brief explanation**, a compact **code example** (shell / Node.js + Mongoose where helpful), a one-line **scenario** when it matters, and a **mini task** you can complete quickly. Practice the mini-tasks and add the small coding questions to a GitHub repo — that will impress interviewers.

Tip: answer clearly in interviews: **(What)** → **(Why it matters)** → **(Example)** → **(Trade-offs/Gotchas)**.

1) What is MongoDB?

Short: A document-oriented NoSQL database that stores JSON-like BSON documents.

Explain: Schema-flexible, horizontally scalable, designed for high performance and easy development. Good for evolving schemas and JSON workloads.

Code (mongo shell):

```
js

// create db & insert
use shop;
db.products.insertOne({name:'mug', price:199, tags:['kitchen']});
```

Scenario: Rapid prototyping or apps with changing data shapes.

Mini task: Insert a document into `test.items` and query it.

2) Documents vs Collections vs Databases

Short: Document = JSON-like record; Collection = group of documents (like table);

Database = namespace of collections.

Explain: Documents are schemaless but can be validated; collections contain heterogeneous documents. Indexes live at collection level.

Code:

```
js

db.users.insertOne({name:'Chetan', age:23});
db.users.find({age: {$gt: 20}});
```

Scenario: Designing data model for users/products.

Mini task: Create `orders` collection and add 3 sample orders.

3) BSON — what and why?

Short: Binary JSON format used internally; supports types JSON doesn't (Date, ObjectId, Binary).

Explain: Efficient storage and type fidelity (e.g., 64-bit ints). ObjectId includes timestamp and uniqueness.

Code:

```
js

const id = ObjectId(); // has embedded timestamp
```

Scenario: Storing binary blobs, dates, or need ObjectId features.

Mini task: Inspect `ObjectId().getTimestamp()` in shell.

4) CRUD basics (create, read, update, delete)

Short: `insertOne/Many`, `find/findOne`, `updateOne/Many`, `deleteOne/Many`.

Explain: Updates can use atomic update operators like `$set`, `$inc`, `$push`. `find()`

returns a cursor.

Code:

```
js

db.products.updateOne({name:'mug'}, {$inc:{stock: -1}});
db.products.find({price: {$lt: 500}}).toArray();
```

Scenario: Implementing product inventory operations.

Mini task: Update one doc with `$set` and query to confirm.

5) Indexes — why and types

Short: Indexes speed reads by avoiding full collection scans. Types: single-field, compound, multikey, text, hashed, TTL.

Explain: Indexes speed reads but slow writes and use memory/disk. Choose fields used in filters/sorts. Use explain plans to verify.

Code:

```
js

db.users.createIndex({email:1}, {unique:true});
db.logs.createIndex({createdAt:1}, {expireAfterSeconds: 60*60*24});
```

Scenario: High-read endpoints or unique constraints (email).

Mini task: Create a compound index on `{userId:1, createdAt:-1}` and explain a query.

6) Explain query planner & `explain()`

Short: `explain()` shows how MongoDB will execute a query (index usage, COLLSCAN).

Explain: Use `explain('executionStats')` to see execution time and number of scanned docs to optimize queries.

Code:

```
js

db.orders.find({userId: 123}).explain('executionStats');
```

Scenario: Optimize slow queries in production.

Mini task: Run `explain` for a slow query and identify if it's a COLLSCAN.

7) Aggregation framework basics

Short: Pipeline-based data processing (`$match` , `$group` , `$project` , `$sort` , `$lookup`).

Explain: Powerful for transformations, grouping, joins (via `$lookup`), and analytics inside DB. Pipelines run on server and can be optimized with indexes for early `$match` / `$sort` .

Code:

```
js

db.sales.aggregate([
  {$match: {status:'shipped'}},
  {$group: {_id: '$productId', totalSold: {$sum: '$qty'}}},
  {$sort: {totalSold:-1}},
  {$limit: 5}
]);
```

Scenario: Generate monthly sales reports without moving data out.

Mini task: Aggregate to compute average `price` per `category` .

8) `$lookup` (joins)

Short: Performs left-outer joins between collections inside aggregation.

Explain: Useful for combining related documents (e.g., orders + users). For large datasets consider denormalization or pipeline tuning.

Code:

```
js

db.orders.aggregate([
  {$lookup:{from:'users', localField:'userId', foreignField:'_id', as:'user'}},
  {$unwind:'$user'}
]);
```

Scenario: Show order with customer details in one query.

Mini task: Use `$lookup` to join `comments` with `posts` .

9) Schema design — embedding vs referencing

Short: Embed related data for fast reads and locality; reference for large/ growing arrays or shared resources.

Explain: Embedding avoids joins and is good for one-to-few relationships; reference for many-to-many or huge arrays. Consider update patterns and document size limit (16MB).

Example:

```
js

// Embedding
post = {title:" ", comments: [{by:'Alice', text:'...'}]}

// Referencing
post = {title:" ", commentIds: [ObjectId('...')]}
```

Scenario: Choose embedding for user profile with small address object, referencing for posts with millions of comments.

Mini task: Model `user` + `addresses` as embedded docs and create an example.

10) Transactions — multi-document ACID

Short: MongoDB supports multi-document ACID transactions (replica set or sharded clusters).

Explain: Use transactions when multiple documents across collections must be updated atomically; they have performance cost — prefer single-document atomic updates where possible.

Code (Node.js):

```
js

const session = client.startSession();
await session.withTransaction(async () => {
  await coll1.insertOne({...}, {session});
  await coll2.updateOne({...}, {$inc:{...}}, {session});
});
session.endSession();
```

Scenario: Financial transfers that must update two accounts atomically.

Mini task: Write a small transaction transferring amount between two accounts.

11) Replication & Replica Sets

Short: Replica set = group of mongod instances providing redundancy and failover (primary + secondaries).

Explain: Primary handles writes, secondaries replicate oplog; use read preferences to read from secondaries if eventual consistency acceptable. Configure arbiter if odd voting needed.

Scenario: Ensure high availability and automatic failover.

Mini task: Explain (in 2 lines) why you wouldn't read from secondaries for real-time inventory counts.

12) Sharding — horizontal scaling

Short: Sharding distributes data across shards by a shard key to scale writes & storage.

Explain: Pick shard key carefully (cardinality, distribution, monotonicity). Hashed keys help distribution; ranged keys help range queries. Sharding adds complexity (balancer, chunk migration).

Scenario: Massive collection (billions of docs) needing horizontal scale.

Mini task: List three properties of a good shard key.

13) Replica lag & read preferences

Short: Replica lag = delay in replication; read preferences (`primary` , `secondary` , `primaryPreferred` , etc.) control where reads go.

Explain: Reading from secondaries may return stale data. Use `readConcern` and `writeConcern` to tune consistency guarantees.

Mini task: Show how to use `readPreference` in a MongoDB connection string (conceptually).

14) `writeConcern` & `readConcern`

Short: `writeConcern` controls acknowledgment level for writes (e.g., `w:1` , `w:majority`); `readConcern` controls isolation (e.g., `local` , `majority`).

Explain: Stronger levels increase durability but may increase latency. Choose appropriate levels for business requirements.

Mini task: In Node.js driver, show `collection.insertOne(doc, {w: 'majority'})` usage.

15) Data validation & schema validation

Short: Use JSON Schema validation at collection level to enforce structure. Tools like Mongoose provide schema at app layer.

Explain: Server-side validation prevents malformed data; combine DB and app validation for safety.

Code:

```
js

db.createCollection('users', {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["email"],
    properties: { email: {bsonType: "string"} }
  }}
});
```

Mini task: Add validator requiring `price` to be number ≥ 0 .

16) TTL indexes (expireAfterSeconds)

Short: TTL indexes remove documents after a certain time based on a date field.

Explain: Useful for sessions, caches, or logs. Deletions happen via background thread — not real-time exact.

Code:

```
js

db.sessions.createIndex({lastAccessed:1}, {expireAfterSeconds: 3600});
```

Scenario: Auto-expire login sessions after 1 hour.

Mini task: Create TTL index and insert a doc with a `Date` field to test.

17) Text search & text indexes

Short: Create a text index and use `$text` queries for basic full-text search (language aware).

Explain: Good for simple search; for advanced search consider Atlas Search or external engines (Elasticsearch). Use weights for boosting fields.

Code:

```
js

db.articles.createIndex({title:'text', body:'text'}, {weights:{title:5}});
db.articles.find({$text: {$search: "mongodb indexing"}});
```

Mini task: Create a text index and run a `$text` query.

18) Geospatial queries

Short: MongoDB supports geospatial indexes (`2dsphere`) and queries like `$near` , `$geoWithin` .

Explain: Good for location-based features (nearby restaurants). Store as GeoJSON.

Code:

```
js

db.places.createIndex({location: '2dsphere'});
db.places.find({location: {$near: {$geometry: {type:'Point', coordinates:[lon,lat]},
$maxDistance:5000}}});
```

Mini task: Insert a point and query for locations within 5km.

19) Aggregation performance tips

Short: `$match` early, use indexes for `$sort` / `$match` , avoid `$unwind` on huge arrays unless necessary, and pipeline optimization.

Explain: Use `$project` to reduce fields early; pipeline stages that reduce docs should be earlier. Use `allowDiskUse:true` when memory-heavy.

Mini task: Rewrite a pipeline to push `$match` to the first stage.

20) Backup & restore (mongodump/mongorestore / Atlas snapshots)

Short: `mongodump` / `mongorestore` for logical backups or use cloud-managed snapshots (Atlas). Test restores regularly.

Explain: Backups must consider oplog/replay and consistency for sharded clusters. Plan RTO/RPO.

Mini task: Describe steps to restore a single collection from a `mongodump`.

21) Authentication & authorization (roles)

Short: MongoDB supports SCRAM auth, LDAP, and role-based access control (DB roles & built-in roles).

Explain: Principle of least privilege: grant only required permissions to services. Use TLS for encrypted connections.

Mini task: Create a read-only user on `reports` DB (`db.createUser({...roles: [{role: 'read', db: 'reports'}]})`).

22) Monitoring & metrics (MMAP/WT, WiredTiger)

Short: Use MongoDB monitoring tools (`mongostat`, `mongotop`) and metrics (WiredTiger cache usage). WiredTiger is default storage engine.

Explain: Watch page faults, lock percentage, replication lag, index miss ratio. Use Atlas or MMS for hosted monitoring.

Mini task: Run `mongostat` (conceptually) and name two metrics you'd check for slow performance.

23) Common pitfalls & anti-patterns

Short: Large unbounded arrays, unindexed queries, monolithic documents near 16MB, wrong shard key, frequent small updates without `$inc` /positional operator misuse.

Explain: Recognize these and prefer bounded arrays, proper indexing, and update operators to modify fields atomically.

Mini task: Identify why storing millions of comments embedded in a post is a bad idea.

24) ODMs vs native driver (Mongoose vs native)

Short: Mongoose provides schema, middleware, and validation at app layer; native driver is lightweight and closer to DB behavior.

Explain: Use Mongoose for rapid app-level schema/validation; prefer native driver for maximum performance or when avoiding abstraction. Watch Mongoose queries vs aggregation parity.

Code (Mongoose example):

js

```
const UserSchema = new mongoose.Schema({name:String, email:{type:String, unique:true}});
const User = mongoose.model('User', UserSchema);
await User.create({name:'Chetan', email:'c@example.com'});
```

Mini task: Create a Mongoose schema with a pre-save hook that lowercases email.

25) Migrating schema & versioning data

Short: Use migration tools (migrate-mongo, custom scripts) and include version field in documents or maintain a migration log.

Explain: Because MongoDB is schemaless, you must migrate existing documents for new app expectations. Test migrations safely and make them idempotent.

Mini task: Write a small script that adds `isActive:true` to all users missing that field.

Quick “small coding question” set (practice)

1. Insert 100 sample `products` and write an aggregation to find top 5 categories by sales.
2. Create a unique index on `username` and show handling of duplicate insert error.
3. Implement a paginated `GET /posts?page=2&limit=10` using `_id` or `createdAt` for efficient pagination.
4. Use `$group` to compute monthly revenue from `orders` collection.
5. Model comments as a referenced collection and write a query to fetch a post with its first 10 comments (use `$lookup` + `$limit`).
6. Demo a transaction in Node.js that moves money between accounts (with rollback on failure).
7. Create a TTL index for `sessions` collection and verify document expiry.

8. Build an index to support `find({status: 'open'}).sort({priority: -1, createdAt: 1})` efficiently.
 9. Show how to update a nested array item using positional operator (`$ / $[<identifier>]`).
 10. Write a migration script to rename field `fullname` → `name` across collection.
-

How to use this sheet for Capgemini Exceller interview

1. **Implement 6–8 small tasks** above and push to GitHub (include short README for each). Interviewers like to see real examples.
 2. **Be ready to explain trade-offs:** embedding vs referencing, shard key choices, `writeConcern` vs latency.
 3. **Practice 2-minute explanations** for replication, sharding, and aggregation pipelines.
 4. **During interviews**, draw simple diagrams for replica sets or sharded clusters, and mention monitoring/backups and production readiness.
-

If you want, I can now:

- Convert this into a **one-page printable cheat sheet**, or
- Generate **complete working code** (Node.js scripts + Mongoose + README + tests) for any **5** of the small coding tasks above (pick which 5).

Which would you like next?